PRD: GitMash

Product Name

GitMash

Domain

gitmash.com

One-Liner

GitMash lets users paste 2 to 3 related GitHub repo URLs, describe what they want to preserve from each, and uses AI agents to analyze, harmonize, test, review, document, and generate one stronger unified project.

Product Vision

Builders often create multiple versions of the same idea across different GitHub repos. One repo may have the best frontend. Another may have a better backend. Another may have the right architecture but weak implementation. GitMash turns that scattered repo history into a structured AI-assisted build process.

The user should not need to manually compare repos, copy files, reconcile dependencies, or explain everything to an AI coding agent from scratch.

GitMash should:

1. Accept multiple GitHub repo URLs.

2. Ask the user what they like from each repo in natural language.

3. Ingest and analyze each repo deeply.

4. Identify the best ideas, code, architecture, features, and patterns.

5. Detect conflicts, weaknesses, security issues, and technical debt.

6. Recommend a unified architecture.

7. Generate a new harmonized project.

8. Add tests, review checkpoints, dev logs, and documentation throughout the build.

9. Produce a final repo that another developer or AI agent can continue without confusion.

Core Product Principle

GitMash is not simply a code merger.

GitMash is an AI repo synthesis system.

The system must not blindly smash codebases together. It must understand, compare, decide, document, build, test, and review.

---

1. Problem Statement

Many developers, founders, and AI builders create several GitHub repos around the same concept. This happens because they:

Start over when they get a better idea.

Use different frameworks across attempts.

Build partial features in separate repos.

Experiment with UI in one repo and backend in another.

Lose track of what is valuable in older versions.

Have multiple AI-generated repos that overlap but do not integrate cleanly.

Current AI coding tools are good at editing a single repo, but they are not optimized for comparing multiple related repos and producing a unified future version.

GitMash solves this by providing a simple GitIngest-like input flow, then running a structured AI analysis and build pipeline.

---

2. Target Users

Primary User

AI-assisted builder / founder

A person who has several half-finished GitHub projects and wants to consolidate them into one better project.

Example:

> “I built three versions of a dashboard. The first has the best auth, the second has the best UI, and the third has the best AI workflow. I want one clean app that keeps the best parts.”

Secondary Users

Developer teams

Teams that have duplicate internal tools, old prototypes, forked experiments, or competing repo versions.

Consultants and agencies

Teams that inherit several codebases from clients and need to quickly determine what should be preserved, rewritten, or discarded.

AI coding power users

People using Codex, Cursor, Claude Code, Windsurf, Replit Agent, or other AI coding tools who want a better planning and handoff layer before coding.

---

3. User Experience Summary

The experience should feel as simple as GitIngest.

Basic Flow

1. User lands on gitmash.com.

2. User pastes 2 to 3 GitHub repo URLs.

3. User optionally connects GitHub for private repos.

4. User writes a natural language instruction describing what they like from each repo.

5. User clicks Analyze Repos.

6. GitMash ingests each repo and builds a structured repo profile.

7. GitMash shows a comparison dashboard.

8. GitMash recommends a unified build plan.

9. User reviews and approves the plan.

10. GitMash starts the build.

11. GitMash runs implementation steps, unit tests, code reviews, and dev logs.

12. User receives a new unified repo, downloadable zip, and documentation pack.

---

4. MVP Scope

MVP Goal

The MVP should allow a user to paste 2 to 3 public GitHub repos, describe the desired outcome, run a deep repo analysis, approve a merge plan, and generate a new project scaffold with selected code, documentation, tests, and build logs.

MVP Should Include

1. Public GitHub repo URL input.

2. Support for 2 to 3 repos per project.

3. Natural language project brief.

4. Git-style repo ingestion.

5. File filtering and markdown/code digest generation.

6. Per-repo analysis.

7. Cross-repo comparison.

8. Feature extraction based on user preferences.

9. Recommended base architecture.

10. Merge plan with keep, rewrite, discard, and create decisions.

11. Human approval checkpoint before build.

12. New project generation.

13. Unit test generation.

14. AI code review pass.

15. Dev logs.

16. Final export as downloadable zip.

17. Final markdown documents:

README.md

PROJECT_SPEC.md

ARCHITECTURE.md

MERGE_PLAN.md

DECISIONS.md

TESTING.md

DEVLOG.md

AGENT_HANDOFF.md

MVP Should Not Include Yet

1. Full private repo OAuth.

2. Automatic production deployment.

3. Full multi-user team workspaces.

4. Complex monorepo management.

5. Automatic CI/CD setup across every possible stack.

6. Guaranteed perfect code merging across unrelated languages.

7. Autonomous pushing to GitHub without user approval.

---

5. Product Positioning

Tagline Options

Primary tagline:

> Turn scattered GitHub repos into one better project.

Other options:

Merge your best repo ideas into one clean build.

AI-powered repo synthesis for builders with too many prototypes.

Paste repos. Describe the goal. Get one harmonized project.

From repo sprawl to production-ready clarity.

What GitMash Is

A multi-repo AI analysis tool.

A repo synthesis engine.

A merge-planning system.

A build assistant.

A documentation and handoff generator.

What GitMash Is Not

Not a basic Git merge tool.

Not just a repo-to-markdown converter.

Not just a code generator.

Not a tool that blindly combines files.

Not a replacement for review, tests, and architecture judgment.

---

6. Core User Flow

Screen 1: Landing Page

Purpose

Explain the product quickly and let the user begin immediately.

Main CTA

Mash Repos

Input Section

The homepage should include a simple input card:

Repo 1 URL: [_____]
Repo 2 URL: [_____]
Repo 3 URL: [optional_____]

Describe what you want GitMash to preserve, combine, or improve:

[Large natural language text box]

[Analyze Repos]

Example Placeholder Text

Example:
Repo 1 has the best backend and database structure.
Repo 2 has the best UI and dashboard layout.
Repo 3 has the best AI workflow concept.
Create one unified Next.js app that keeps the best architecture, rewrites weak pieces, and documents the build clearly.

Screen 2: Repo Validation

After the user submits URLs, GitMash validates each repo.

Validation Checks

For each repo:

1. Is the URL valid?

2. Is it a GitHub repo?

3. Is it public or private?

4. Can it be cloned?

5. What is the default branch?

6. What is the repo size?

7. What language/framework appears to be used?

8. Does it contain dangerous or unsupported files?

9. Does it contain package manifests?

10. Does it contain tests?

Output

Show each repo as a card.

Repo A: my-dashboard-v1
Status: Valid
Branch: main
Stack Detected: Next.js, TypeScript, Tailwind
Size: 2.4 MB
Tests Detected: No

Repo B: my-dashboard-v2
Status: Valid
Branch: main
Stack Detected: Next.js, Supabase, Tailwind
Size: 4.1 MB
Tests Detected: Yes

User Action

Button:

Start Analysis

Screen 3: Natural Language Intent Confirmation

Before running the full analysis, GitMash should convert the user's natural language instructions into a structured intent profile.

Example User Input

I like the UI from repo 2, the auth from repo 1, and the AI workflow from repo 3. I want the final project to be clean, modern, and easy to continue building with AI coding agents.

GitMash Structured Interpretation


```
{
  "desired_final_product": "A clean unified application combining the strongest UI, auth, and AI workflow elements from the submitted repos.",
  "preserve_preferences": [
    {
      "source_repo": "repo_2",
      "preference": "Preserve UI and dashboard layout"
    },
    {
      "source_repo": "repo_1",
      "preference": "Preserve authentication approach"
    },
    {
      "source_repo": "repo_3",
      "preference": "Preserve AI workflow concept"
    }
  ],
  "quality_preferences": [
    "clean architecture",
    "modern implementation",
    "easy for AI coding agents to continue",
    "well documented",
    "unit tested"
  ],
  "implicit_constraints": [
    "avoid blindly merging duplicated code",
    "standardize stack choices",
    "document all major decisions"
  ]
}
```


User Action

User can edit the interpreted intent before continuing.

Buttons:

Looks Good

Edit Instructions

---

7. Analysis Pipeline

This is the most important part of GitMash.

The pipeline must be layered. GitMash should never feed several full repos into one giant prompt and ask an AI to “combine them.” That would create inconsistent, fragile output.

Instead, GitMash needs a deterministic plus AI-assisted pipeline.

Pipeline Overview

1. Repo Intake
2. Repo Cloning
3. File Filtering
4. Static Analysis
5. Digest Generation
6. Per-Repo Summarization
7. User Intent Extraction
8. Feature Inventory
9. Cross-Repo Comparison
10. Conflict Detection
11. Best Practice Audit
12. Architecture Recommendation
13. Merge Plan Generation
14. Human Review Gate
15. Build Task Planning
16. Implementation
17. Unit Testing
18. Code Review
19. Repair Loop

20. Documentation and Dev Logs
21. Final Export

---

8. Detailed Pipeline Requirements

8.1 Repo Intake

Goal

Collect repo URLs and user instructions.

Inputs

2 required GitHub repo URLs.

1 optional GitHub repo URL.

Natural language user brief.

Optional final project name.

Optional preferred stack.

Optional files/features to exclude.

Optional target output type:

Analysis only

Build plan

New project scaffold

Full generated repo

Requirements

Validate all URLs before cloning.

Reject unsupported hosts for MVP unless explicitly planned.

Display clear error messages.

Allow users to remove or replace a repo before analysis.

8.2 Repo Cloning

Goal

Fetch source repos into a controlled temporary environment.

Requirements

Clone repos into isolated build workspace.

Use shallow clone by default.

Avoid pulling unnecessary Git history in MVP.

Record:

repo URL

branch

latest commit SHA

clone timestamp

repo size

detected primary language

Never execute repo code during initial analysis.

Do not run install scripts during intake.

Security Requirements

Treat repo contents as untrusted input.

Never execute arbitrary scripts from the repo without sandboxing.

Strip or mask secrets before sending content to AI models.

Detect common secret patterns:

API keys

tokens

private keys

database URLs

.env files

credentials

Exclude sensitive files from AI prompts unless user explicitly allows safe redacted handling.

8.3 File Filtering

Goal

Avoid noise and prevent the AI from analyzing irrelevant generated files.

Default Excluded Paths

```
.git/
node_modules/
vendor/
dist/
build/
.next/
.nuxt/
```

out/
coverage/
.cache/
.turbo/
.parcel-cache/
.DS_Store
*.lock when not needed for dependency analysis
*.png
*.jpg
*.jpeg
*.gif
*.webp
*.mp4
*.mov
*.zip
*.tar
*.gz
.env
.env.*

Default Included Files

README*
package.json
tsconfig.json
next.config.*
vite.config.*
tailwind.config.*
src/**
app/**
pages/**
components/**
lib/**
server/**
api/**
routes/**
controllers/**
models/**
schemas/**
prisma/**
supabase/**
db/**
tests/**
__tests__/**

docs/**
requirements.txt
pyproject.toml
Dockerfile
docker-compose.yml

Requirement

The file filtering rules should be user-configurable later, but hardcoded defaults are acceptable for MVP.

8.4 Static Analysis

Goal

Extract objective facts from each repo before asking AI to reason.

Static Analysis Should Detect

1. Project type

Next.js

Vite

React

Express

FastAPI

Flask

Django

Node CLI

Python package

Unknown

2. Language

TypeScript

JavaScript

Python

Go

Rust

Mixed

3. Package manager

npm

yarn

pnpm

bun

pip

poetry

4. Dependency profile

UI libraries

Auth providers

Database libraries

AI SDKs

Testing libraries

Build tools

5. App structure

routes

pages

API endpoints

components

services

models

database schemas

environment variables

config files

6. Testing profile

Existing unit tests

Existing integration tests

Existing E2E tests

Test command

Coverage config

7. Documentation profile

README quality

docs folder

architecture docs

setup instructions

missing docs

8. Risk profile

missing tests

deprecated dependencies

untyped files

hardcoded secrets

duplicate code

overly complex files

no clear entry point

8.5 Digest Generation

Goal

Convert each repo into AI-readable structured artifacts.

GitMash should not only create one giant markdown dump. It should create structured artifacts.

Required Artifacts Per Repo

```
repo-summary.json
file-tree.md
file-tree.json
dependency-analysis.json
route-map.json
component-map.json
data-model-map.json
test-map.json
risk-report.json
repo-digest.md
important-files/
  file-001.md
  file-002.md
  file-003.md
```

repo-digest.md Structure

Repo Digest

Repo Metadata

Detected Stack

Package Scripts

Dependency Summary

File Tree

Key Source Files

Routes and API Endpoints

Components

Services

Database and Data Models

Auth

AI Features

Tests

Documentation

Potential Risks

Initial Static Analysis Notes

8.6 Per-Repo Summarization Agent

Goal

Have AI analyze each repo independently before cross-repo comparison.

Agent Output Schema

```
{
  "repo_id": "repo_a",
  "repo_name": "example-repo",
  "primary_purpose": "string",
  "detected_stack": ["string"],
  "main_features": ["string"],
  "best_parts": [
    {
      "name": "string",
      "type": "component | service | route | schema | workflow | design | documentation | test",
      "path": "string",
      "why_it_is_good": "string",
      "confidence": 0.0
    }
  ],
  "weak_parts": [
    {
      "name": "string",
      "path": "string",
      "problem": "string",
      "recommendation": "keep | rewrite | discard",
      "confidence": 0.0
    }
  ],
  "technical_debt": ["string"],
  "security_concerns": ["string"],
```

```
  "testing_gaps": ["string"],
  "documentation_gaps": ["string"],
  "recommended_role_in_final_build": "base_repo | feature_source | concept_source |
discard_most | unclear"
}
```

8.7 User Intent Extraction Agent

Goal

Convert the user’s natural language instructions into structured build guidance.

Example Input

Repo 1 has the best database and auth. Repo 2 has a better landing page and dashboard. Repo 3 has the AI agent flow I want. Keep the final project as Next.js and make it easy for another AI agent to continue.

Output Schema

```
{
  "final_project_goal": "string",
  "preferred_stack": ["string"],
  "source_preferences": [
    {
      "repo_id": "repo_a",
      "items_to_preserve": ["string"],
      "items_to_avoid": ["string"],
      "notes": "string"
    }
  ],
  "quality_bar": {
    "testing_required": true,
    "documentation_required": true,
    "dev_logs_required": true,
    "code_reviews_required": true,
    "future_agent_handoff_required": true
  },
  "explicit_constraints": ["string"],
  "implicit_constraints": ["string"],
  "unknowns_to_resolve": ["string"]
```

}

8.8 Feature Inventory Agent

Goal

Create a normalized feature map across all repos.

Example Output

```
{
  "features": [
    {
      "feature_name": "Authentication",
      "present_in_repos": ["repo_a", "repo_b"],
      "implementations": [
        {
          "repo_id": "repo_a",
          "paths": ["lib/auth.ts", "app/login/page.tsx"],
          "quality_score": 8,
          "notes": "Uses Supabase auth and has cleaner session handling."
        },
        {
          "repo_id": "repo_b",
          "paths": ["src/auth"],
          "quality_score": 5,
          "notes": "Partially implemented and poorly documented."
        }
      ],
      "recommended_decision": "Use repo_a implementation as base."
    }
  ]
}
```

Required Feature Categories

Auth

Routing

Dashboard

UI components

Data models

API layer

AI/LLM workflows

State management

File upload

Database integration

Payment integration

Admin tools

Tests

Documentation

Deployment setup

8.9 Cross-Repo Comparison Agent

Goal

Compare the repos and determine how they relate.

Questions It Must Answer

1. Are the repos actually related?

2. What is the shared product goal?

3. Which repo is the best base?

4. Which repo has the best UI?

5. Which repo has the best backend?

6. Which repo has the cleanest architecture?

7. Which repo has the best documentation?

8. Which repo has the best tests?

9. Which repo has the most technical debt?

10. Which features overlap?

11. Which implementations conflict?

12. What should be preserved, rewritten, discarded, or newly created?

Output


```
{
  "shared_product_goal": "string",
  "repo_relationship": "highly_related | somewhat_related | weakly_related | unrelated",
  "recommended_base_repo": {
    "repo_id": "repo_a",
    "reason": "string",
    "confidence": 0.0
  },
  "repo_roles": [
    {
      "repo_id": "repo_a",
      "role": "base architecture",
```

```
      "reason": "string"
    },
    {
      "repo_id": "repo_b",
      "role": "UI source",
      "reason": "string"
    }
  ],
  "feature_decisions": [
    {
      "feature": "string",
      "decision": "keep_from_repo | rewrite | discard | create_new",
      "source_repo": "repo_a",
      "reason": "string",
      "target_location": "string"
    }
  ],
  "conflicts": [
    {
      "conflict_area": "auth | database | routing | styling | package_manager | framework |
state_management | testing",
      "repo_a_approach": "string",
      "repo_b_approach": "string",
      "recommended_resolution": "string",
      "reason": "string"
    }
  ]
}
```

8.10 Best Practice Audit Agent

Goal

Evaluate each repo and the proposed final architecture against best practices.

Audit Categories

1. Architecture

clear separation of concerns

modularity

maintainability

framework alignment

file organization

2. Security

secrets handling

auth safety

dependency risk

unsafe eval or shell execution

API exposure

input validation

3. Testing

unit tests

integration tests

E2E tests

testable architecture

mocking strategy

4. Performance

bundle size risk

unnecessary dependencies

inefficient rendering

slow API routes

data fetching patterns

5. Accessibility

semantic HTML

keyboard navigation

labels

contrast warnings where detectable

6. Developer experience

setup clarity

scripts

type safety

linting

formatting

documentation

7. Future AI-agent compatibility

clear docs

small files

explicit decisions

dev logs

task breakdowns

handoff notes

Output

```
{
  "overall_score": 0,
  "architecture_score": 0,
  "security_score": 0,
  "testing_score": 0,
  "documentation_score": 0,
  "futureproofing_score": 0,
  "risks": [
    {
      "severity": "low | medium | high | critical",
      "category": "string",
      "description": "string",
      "recommended_fix": "string"
    }
  ],
  "required_before_build": ["string"],
  "recommended_after_build": ["string"]
}
```

8.11 Architecture Recommendation Agent

Goal

Design the future unified project.

It Must Decide

1. Final app framework.

2. Package manager.

3. Folder structure.

4. Auth approach.

5. Database approach.

6. API approach.

7. UI component strategy.

8. State management.

9. Testing strategy.

10. Documentation strategy.

11. Which repo becomes the base.

12. Which files are copied.

13. Which files are rewritten.

14. Which concepts are preserved but reimplemented.

15. Which files are discarded.

Output

Unified Architecture Recommendation

Final Product Goal

Recommended Base Repo

Final Stack

Why This Stack

Folder Structure

Features to Preserve

Features to Rewrite

Features to Discard

New Features to Create

Dependency Harmonization Plan

Routing Plan

Data Model Plan

Auth Plan

Testing Plan

Documentation Plan

Risks and Mitigations

Build Milestones

---

9. Merge Plan

Before GitMash builds anything, it must produce a clear merge plan.

Merge Plan Categories

Every meaningful source file or feature should fall into one of these categories:

1. Keep

Use mostly as-is.

2. Adapt

Reuse but modify for final architecture.

3. Rewrite

Preserve the idea but rewrite implementation.

4. Discard

Do not include in final build.

5. Create New

Needed for final project but not present in source repos.

Example Merge Plan

GitMash Merge Plan

Base Repo

Use `repo_2` as the base because it has the cleanest Next.js structure, strongest routing, and best dashboard layout.

Keep

- `components/DashboardShell.tsx` from repo_2
- `lib/supabase/client.ts` from repo_1
- `docs/setup.md` from repo_3

Adapt

- `app/login/page.tsx` from repo_1
  - Adapt styling to match repo_2 dashboard design.

- `lib/ai/workflow.ts` from repo_3
  - Convert into typed service under `src/lib/ai`.

Rewrite

- AI workflow execution
  - Preserve repo_3 concept.
  - Rewrite with typed interfaces, better error handling, and test coverage.

Discard

- Old CSS files from repo_1
- Duplicate dashboard pages from repo_3
- Deprecated API route structure from repo_2

Create New

- `ARCHITECTURE.md`
- `TESTING.md`
- `src/lib/logger.ts`
- `src/types/project.ts`
- Unit tests for auth, AI workflow, and dashboard utilities

---

10. Human Review Gate

GitMash must not immediately build after analysis.

It should show the user:

1. Repo summaries.

2. Feature comparison.

3. Recommended base repo.

4. Merge plan.

5. Risks.

6. Proposed final architecture.

7. Estimated build phases.

Then the user can choose:

Approve and Build

Edit Instructions

Change Base Repo

Exclude Feature

Download Analysis Only

---

11. Build Pipeline

Once the user approves the plan, GitMash starts the build.

Build Pipeline Overview

1. Create isolated build workspace
2. Initialize final project
3. Apply selected base repo
4. Copy/adapt selected files
5. Rewrite selected concepts
6. Harmonize dependencies
7. Generate missing glue code
8. Generate tests
9. Run tests
10. Run lint/typecheck
11. Run code review agent
12. Fix issues
13. Repeat validation
14. Generate docs and dev logs
15. Package final output

---

12. Build Requirements

12.1 Workspace Creation

GitMash should create a new isolated workspace for the final project.

Required directory structure during build:

```
gitmash-workspace/
  source-repos/
    repo-a/
    repo-b/
    repo-c/
  analysis/
    repo-a/
    repo-b/
    repo-c/
    cross-repo/
  output/
    final-project/
  logs/
    build-run.md
    agent-events.jsonl
```

```
  reviews/
    review-001.md
    review-002.md
  test-results/
```

12.2 Final Project Structure

For a typical Next.js output project:

```
final-project/
  README.md
  PROJECT_SPEC.md
  ARCHITECTURE.md
  MERGE_PLAN.md
  DECISIONS.md
  TESTING.md
  AGENT_HANDOFF.md
  DEVLOG.md
  package.json
  tsconfig.json
  next.config.ts
  src/
    app/
    components/
    lib/
    services/
    types/
    tests/
  docs/
    devlog/
    decisions/
    reviews/
```

For non-Next.js repos, structure should adapt to detected stack.

12.3 Dependency Harmonization

GitMash must not simply concatenate package.json dependencies.

It must:

1. Choose one package manager.

2. Choose compatible versions.

3. Remove duplicate libraries.

4. Prefer the base repo’s dependency versions unless another repo has a clear reason to upgrade.

5. Detect incompatible framework versions.

6. Avoid adding unused dependencies.

7. Generate a dependency decision log.

Required Output

Dependency Decisions

Package Manager

Selected: pnpm

Reason:
Repo 2 already uses pnpm and has the cleanest dependency setup.

Preserved Dependencies

- next
- react
- react-dom
- tailwindcss
- @supabase/supabase-js

Removed Dependencies

- old-auth-library
  - Reason: replaced by Supabase auth.

Added Dependencies

- zod
  - Reason: used for input validation and safer API boundaries.

12.4 Code Generation Standards

Generated code must follow these rules:

1. Prefer small files.

2. Use typed interfaces where applicable.

3. Avoid hidden magic.

4. Add comments only where useful.

5. Use clear naming.

6. Avoid large monolithic components.

7. Keep business logic separate from UI.

8. Add error handling.

9. Add testable helper functions.

10. Avoid adding unnecessary dependencies.

11. Do not preserve broken code simply because it existed in a source repo.

12. Document major design decisions.

---

13. Testing Requirements

GitMash must set a high standard for tests.

Test Strategy

Each generated project should include at least:

1. Unit tests for pure functions.

2. Unit tests for service logic.

3. Component tests where practical.

4. Basic integration tests for key workflows.

5. Smoke test for app startup or core entrypoint.

6. Mocking strategy for external services.

For Next.js / TypeScript Projects

Recommended default test stack:

Vitest
React Testing Library
Playwright optional for E2E

For Python Projects

Recommended default test stack:

pytest
pytest-asyncio if needed
httpx for API tests if FastAPI

Required Test Artifacts

TESTING.md
test-results/
coverage-summary.md when available

Test Generation Rules

GitMash should generate tests for:

1. Newly created utility functions.

2. Adapted services.

3. AI workflow logic.

4. Auth helpers where mockable.

5. API route handlers where practical.

6. Data transformation functions.

7. Any file that contains critical business logic.

Test Run Requirements

During build, GitMash should run:

install
lint
typecheck
unit tests
build

Where commands are stack-specific.

Example for Next.js:

pnpm install
pnpm lint
pnpm typecheck
pnpm test
pnpm build

If a repo does not have these scripts, GitMash should add reasonable scripts when possible.

---

14. AI Code Review Requirements

GitMash should perform code reviews during the build.

Review Stages

Review 1: Post-Architecture Review

Before implementation begins.

Checks:

Is the architecture coherent?

Does it reflect user preferences?

Are source repos used appropriately?

Are there risky assumptions?

Review 2: Mid-Build Review

After initial code generation.

Checks:

Are copied/adapted files integrated cleanly?

Are dependencies aligned?

Are major features missing?

Are there obvious bugs?

Review 3: Test Review

After tests are generated.

Checks:

Do tests cover meaningful behavior?

Are tests shallow or fake?

Are mocks reasonable?

Are important flows untested?

Review 4: Final Review

Before export.

Checks:

Does the final project match the approved merge plan?

Are docs complete?

Are dev logs complete?

Do tests pass?

Are known issues documented?

Code Review Output Format

Code Review

Review Stage

Final Review

Summary

Issues Found

Critical

High

Medium

Low

Required Fixes

Recommended Improvements

Approved for Export

Yes / No

---

15. Repair Loop

If tests, lint, typecheck, build, or code review fail, GitMash should enter a repair loop.

Repair Loop Steps

1. Capture error.

2. Classify error.

3. Identify likely files.

4. Ask implementation agent to patch.

5. Rerun targeted test.

6. Rerun broader validation.

7. Log the fix.

8. Continue until:

validations pass, or

max repair attempts reached.

Max Attempts

Default:

3 repair attempts per failure category

Failure categories:

install failure

lint failure

typecheck failure

unit test failure

build failure

code review critical issue

If Repair Fails

GitMash should not hide the failure.

It should export:

KNOWN_ISSUES.md
failed-test-output.txt
repair-attempts.md

The final output should clearly say:

> This build is incomplete. Here are the remaining issues and suggested next steps.

---

16. Dev Logs and Documentation Requirements

This is a core differentiator for GitMash.

The final project must be easy for future agents and people to continue.

Required Documentation Files

README.md

Must include:

Project overview

Setup instructions

Scripts

Environment variables

How to run tests

How to continue development

Source repos used

PROJECT_SPEC.md

Must include:

Final product goal

User’s original intent

Main features

Non-goals

Future roadmap

ARCHITECTURE.md

Must include:

Final stack

Folder structure

Key modules

Data flow

API design

Auth approach

Dependency decisions

MERGE_PLAN.md

Must include:

Source repo summaries

Base repo decision

Keep/adapt/rewrite/discard/create-new decisions

Rationale

DECISIONS.md

Must include major architecture decisions.

Format:

Decision 001: Use Supabase Auth

Decision

Use Supabase Auth as the final authentication system.

Why

Repo A had the cleanest implementation and the final app already uses Supabase for data.

Alternatives Considered

- Clerk
- Custom auth

Tradeoffs

Supabase is simpler for this MVP but may require additional role-based access control work later.

TESTING.md

Must include:

Test stack

How to run tests

What is covered

What is not covered

Mocking strategy

Known testing gaps

DEVLOG.md

Must include a chronological build summary.

Example:

Dev Log

Build Run

Date:
Source repos:
Final project name:

Step 1: Repo Analysis

Summary:
- Repo A analyzed as backend/auth source.
- Repo B analyzed as UI source.
- Repo C analyzed as AI workflow concept source.

Step 2: Architecture Decision

Decision:
Use Repo B as base.

Reason:
Repo B has the cleanest Next.js structure and strongest dashboard layout.

Step 3: Implementation

Changes:
- Created final project scaffold.
- Added Supabase auth from Repo A.

- Adapted dashboard components from Repo B.
- Rewrote AI workflow concept from Repo C.

Step 4: Testing

Results:
- Unit tests passed.
- Typecheck passed.
- Build failed once, repaired missing import.

Step 5: Final Review

Status:
Approved for export.

AGENT_HANDOFF.md

This is critical.

It should let another AI agent continue the project.

Must include:

Agent Handoff

Current Status

What Was Built

What Was Preserved From Each Repo

What Was Rewritten

What Was Discarded

Known Issues

Next Recommended Tasks

Important Files

How to Safely Continue

Do Not Change Without Reviewing

Suggested Prompts for Future AI Agents

---

17. Agent System

GitMash should use multiple specialized agents instead of one general agent.

Required Agents

1. Intake Agent

Responsibilities:

Validate user input.

Normalize repo URLs.

Extract user intent.

Identify missing details.

2. Repo Ingestion Agent

Responsibilities:

Clone repos.

Filter files.

Generate file tree.

Create repo digest.

3. Static Analysis Agent

Responsibilities:

Detect stack.

Detect dependencies.

Detect tests.

Detect routes/components/models.

Identify objective repo facts.

4. Repo Summary Agent

Responsibilities:

Analyze each repo independently.

Identify strengths and weaknesses.

Identify best candidate files/features.

5. Feature Extraction Agent

Responsibilities:

Map features across repos.

Connect user preferences to actual files.

Identify feature ownership.

6. Comparison Agent

Responsibilities:

Compare repos.

Detect overlap.

Detect conflicts.

Recommend source roles.

7. Merge Architect Agent

Responsibilities:

Choose base repo.

Design final architecture.

Create merge plan.

Create build milestones.

8. Security Agent

Responsibilities:

Detect secrets.

Flag unsafe patterns.

Review auth and API risk.

Ensure secrets are excluded from prompts and logs.

9. Test Architect Agent

Responsibilities:

Design testing strategy.

Generate test plan.

Identify critical test cases.

10. Implementation Agent

Responsibilities:

Create final project.

Copy/adapt/rewrite code.

Update dependencies.

Add tests.

Add docs.

11. Code Review Agent

Responsibilities:

Review generated code.

Review architecture alignment.

Review tests.

Review documentation.

Produce actionable findings.

12. Repair Agent

Responsibilities:

Fix lint, typecheck, test, and build failures.

Log each fix.

Avoid making unrelated changes.

13. Documentation Agent

Responsibilities:

Generate README.

Generate architecture docs.

Generate dev logs.

Generate handoff docs.

---

18. Data Model

Project

```
{
  "id": "string",
  "name": "string",
  "status": "draft | analyzing | awaiting_approval | building | reviewing | completed | failed",
  "created_at": "datetime",
  "updated_at": "datetime",
  "user_brief": "string",
  "structured_intent": {}
}
```

SourceRepo

```
{
  "id": "string",
  "project_id": "string",
  "url": "string",
  "name": "string",
  "owner": "string",
  "branch": "string",
  "commit_sha": "string",
  "clone_status": "pending | cloned | failed",
  "detected_stack": ["string"],
  "size_bytes": 0
}
```

RepoAnalysis

```
{
  "id": "string",
  "source_repo_id": "string",
  "file_tree": {},
  "dependency_analysis": {},
  "route_map": {},
  "component_map": {},
```

```
  "test_map": {},
  "risk_report": {},
  "summary": {}
}
```

FeatureDecision

```
{
  "id": "string",
  "project_id": "string",
  "feature_name": "string",
  "decision": "keep | adapt | rewrite | discard | create_new",
  "source_repo_id": "string",
  "source_paths": ["string"],
  "target_paths": ["string"],
  "reason": "string",
  "confidence": 0.0
}
```

BuildTask

```
{
  "id": "string",
  "project_id": "string",
  "title": "string",
  "description": "string",
  "status": "pending | in_progress | completed | failed",
  "depends_on": ["string"],
  "related_files": ["string"],
  "test_requirements": ["string"]
}
```

TestRun

```
{
  "id": "string",
  "project_id": "string",
  "command": "string",
  "status": "passed | failed | skipped",
  "output_path": "string",
  "created_at": "datetime"
}
```

Review

```
{
  "id": "string",
  "project_id": "string",
  "stage": "architecture | mid_build | test | final",
  "status": "approved | needs_changes | failed",
  "findings": []
}
```

DevLogEntry

```
{
  "id": "string",
  "project_id": "string",
  "step": "string",
  "summary": "string",
  "files_changed": ["string"],
  "decisions": ["string"],
  "errors": ["string"],
  "created_at": "datetime"
}
```

---

19. Functional Requirements

FR-001: Repo URL Input

The user must be able to submit 2 to 3 GitHub repo URLs.

FR-002: Natural Language Brief

The user must be able to describe what they want preserved, combined, rewritten, or improved.

FR-003: Repo Validation

The system must validate repo URLs before analysis.

FR-004: Repo Cloning

The system must clone public repos into an isolated temporary workspace.

FR-005: File Filtering

The system must exclude irrelevant and sensitive files by default.

FR-006: Static Repo Analysis

The system must detect stack, dependencies, routes, tests, docs, and risks.

FR-007: Digest Generation

The system must generate structured markdown and JSON digests for each repo.

FR-008: Per-Repo Summary

The system must generate a structured summary for each repo.

FR-009: Intent Extraction

The system must convert the user brief into structured build intent.

FR-010: Feature Inventory

The system must identify features across all submitted repos.

FR-011: Cross-Repo Comparison

The system must compare repos and identify overlap, conflicts, and strengths.

FR-012: Merge Plan

The system must produce a keep/adapt/rewrite/discard/create-new merge plan.

FR-013: User Approval

The system must require user approval before building.

FR-014: Build Workspace

The system must create a clean output workspace for the final project.

FR-015: Code Generation

The system must generate or adapt source files according to the approved merge plan.

FR-016: Dependency Harmonization

The system must generate one coherent dependency setup.

FR-017: Test Generation

The system must generate unit tests for critical generated/adapted logic.

FR-018: Validation Commands

The system must attempt lint, typecheck, test, and build commands where supported.

FR-019: Code Review

The system must run AI code review before final export.

FR-020: Repair Loop

The system must attempt to repair validation failures.

FR-021: Dev Logs

The system must generate chronological dev logs.

FR-022: Final Documentation

The system must generate README, architecture docs, merge plan, decisions, testing docs, and handoff docs.

FR-023: Export

The system must allow the user to download the final project as a zip.

FR-024: Known Issues

If the final build is incomplete, the system must clearly document remaining issues.

---

20. Non-Functional Requirements

Security

Repo contents must be treated as untrusted.

Do not execute arbitrary repo code during analysis.

Exclude or redact secrets.

Do not store GitHub tokens unencrypted.

Do not expose repo contents between users.

Sanitize logs.

Clearly mark unsupported or risky files.

Reliability

Pipeline steps should be resumable where practical.

Failed steps should produce useful error messages.

Analysis artifacts should persist for debugging.

Build failures should not erase previous analysis.

Performance

MVP should support small to medium repos.

Large repos should trigger a warning.

Analysis should use chunking, not one giant prompt.

File filtering should reduce token usage.

Scalability

Use async background jobs for ingestion and builds.

Store artifacts separately from app database.

Queue long-running work.

Keep agent outputs schema-validated.

Maintainability

Every pipeline step should be modular.

Agent prompts should be versioned.

Output schemas should be validated.

Build logs should be easy to inspect.

Future Agent Compatibility

All decisions should be documented.

Every generated project should include AGENT_HANDOFF.md.

Dev logs should be written in plain English.

Tasks should be explicit and sequenced.

---

21. Suggested Technical Architecture

Frontend

Next.js
TypeScript
Tailwind
shadcn/ui
React Query or TanStack Query

Backend

FastAPI or Next.js API routes
Python preferred for repo analysis and code processing

Recommended architecture:

Frontend: Next.js
Backend API: FastAPI
Workers: Python background workers
Queue: Redis + Celery or RQ
Database: Postgres
Artifact Storage: local filesystem for MVP, S3 later
AI Provider Layer: model-agnostic abstraction
Repo Tools: git CLI, ripgrep, tree-sitter where useful

MVP Simpler Stack

For fastest build:

Next.js app
Server-side API routes
Local filesystem workspaces
Postgres or SQLite
Background job runner
AI provider abstraction

But for a stronger futureproof build, separate frontend, backend, and worker.

---

22. API Endpoints

POST /api/projects

Creates a new GitMash project.

Request:

```
{
  "repo_urls": ["string", "string", "string"],
  "user_brief": "string",
  "final_project_name": "string"
}
```

Response:

```
{
  "project_id": "string",
```

```
  "status": "draft"
}
```

POST /api/projects/:id/validate

Validates repo URLs.

POST /api/projects/:id/analyze

Starts repo ingestion and analysis.

GET /api/projects/:id/status

Returns current pipeline status.

GET /api/projects/:id/analysis

Returns analysis results.

POST /api/projects/:id/approve-plan

Approves the merge plan.

POST /api/projects/:id/build

Starts the build.

GET /api/projects/:id/logs

Returns dev logs and pipeline logs.

GET /api/projects/:id/reviews

Returns code review artifacts.

GET /api/projects/:id/export

Downloads final zip.

---

23. UI Screens

23.1 Home

Includes:

Product explanation

Repo URL inputs

Natural language brief

CTA

23.2 Validation Screen

Includes:

Repo cards

Detected stack

Branch

Warnings

Continue button

23.3 Analysis Progress Screen

Shows pipeline steps:

Cloning repos
Filtering files
Generating digests
Analyzing repo A
Analyzing repo B
Comparing repos
Creating merge plan
Running best practice audit

23.4 Analysis Dashboard

Includes:

Repo summary cards

Feature matrix

Best parts from each repo

Weaknesses

Conflicts

Recommended base repo

Best practice audit

23.5 Merge Plan Review

Includes:

Keep

Adapt

Rewrite

Discard

Create New

Final architecture

Risks

Approval button

23.6 Build Progress

Shows:

Build tasks

Current agent activity

Tests run

Reviews complete

Errors found

Repairs attempted

Dev log entries

23.7 Final Output

Includes:

Download zip

View generated docs

View test results

View code reviews

View known issues

Option to create GitHub repo in future version

---

24. Acceptance Criteria

MVP Acceptance Criteria

The MVP is successful when a user can:

1. Paste 2 public GitHub repo URLs.

2. Add a natural language description of what they like from each repo.

3. Start analysis.

4. Receive per-repo summaries.

5. See a cross-repo feature comparison.

6. See a recommended base repo.

7. See a keep/adapt/rewrite/discard/create-new merge plan.

8. Approve the plan.

9. Generate a new project output folder.

10. Receive generated docs.

11. Receive generated tests.

12. See validation results.

13. Download a final zip.

Build Quality Acceptance Criteria

A generated project must include:

1. README.md

2. PROJECT_SPEC.md

3. ARCHITECTURE.md

4. MERGE_PLAN.md

5. DECISIONS.md

6. TESTING.md

7. DEVLOG.md

8. AGENT_HANDOFF.md

9. At least one meaningful test file if the stack supports testing.

10. A clear KNOWN_ISSUES.md if anything failed.

---

25. Risk Management

Risk: Repos are unrelated

Mitigation:

Detect low similarity.

Warn user.

Ask for clarification.

Offer analysis-only mode.

Risk: Different stacks conflict

Example:

Repo A is Next.js.

Repo B is Django.

Repo C is Flask.

Mitigation:

Recommend one final stack.

Preserve concepts instead of copying incompatible code.

Document tradeoffs.

Risk: AI hallucinates file paths or features

Mitigation:

Use static analysis facts.

Require file path citations in agent outputs.

Validate referenced files exist.

Schema-validate AI outputs.

Risk: Generated project does not build

Mitigation:

Run build checks.

Use repair loop.

Document remaining failures.

Avoid claiming completion if validation fails.

Risk: Sensitive files are included

Mitigation:

Exclude .env files.

Run secret scanning.

Redact before prompts.

Never include secrets in docs or logs.

Risk: Huge repos exceed context limits

Mitigation:

Chunk repo files.

Prioritize important files.

Summarize per module.

Skip generated files.

Warn user for large repos.

Risk: User expects perfect autonomous repo creation

Mitigation:

Show clear pipeline status.

Require approval gate.

Clearly separate analysis, plan, build, and validation.

Document known issues.

---

26. Prompting Standards

All agent prompts should require structured output.

Rules

1. Do not ask an AI to merge entire repos in one prompt.

2. Every agent output must be schema-validated.

3. Every recommendation must cite source files when possible.

4. Every keep/adapt/rewrite decision must include a reason.

5. Every major build decision must be written to DECISIONS.md.

6. Every generated file must map back to a build task.

7. Any uncertainty must be explicitly stated.

8. The AI must not pretend tests passed if they were not run.

---

27. Codex Build Instructions

Use this section directly with Codex.

Build Goal

Build the MVP of GitMash: a web app that lets a user paste 2 to 3 GitHub repo URLs, describe what they want preserved from each, analyze the repos, generate a merge plan, approve it, and produce a new documented project scaffold with tests, reviews, and dev logs.

Build Order

Phase 1: App Scaffold

Create the app foundation.

Tasks:

1. Set up Next.js with TypeScript.

2. Add Tailwind and a clean UI component structure.

3. Create homepage with repo URL inputs and natural language brief.

4. Add project creation API.

5. Add basic project status model.

6. Add local workspace folder structure.

Deliverables:

app/
components/
lib/
server/
types/
README.md

Phase 2: Repo Validation and Cloning

Tasks:

1. Validate GitHub repo URLs.

2. Extract owner and repo name.

3. Clone public repos into isolated workspace.

4. Record branch, commit SHA, and clone status.

5. Add error handling for invalid, private, or unavailable repos.

Tests:

Valid GitHub URL parser.

Invalid URL rejection.

Repo metadata extraction.

Workspace path generation.

Phase 3: File Filtering and Static Analysis

Tasks:

1. Implement file tree walker.

2. Apply default exclusion rules.

3. Detect project stack.

4. Detect package manager.

5. Parse package.json.

6. Detect tests.

7. Detect docs.

8. Generate file-tree.json.

9. Generate dependency-analysis.json.

Tests:

Exclusion rule tests.

Stack detection tests.

Package manager detection tests.

File tree generation tests.

Phase 4: Repo Digest Generation

Tasks:

1. Generate repo-digest.md for each repo.

2. Generate structured artifact files.

3. Chunk important source files.

4. Save artifacts under analysis/repo-id.

Tests:

Digest includes expected sections.

Sensitive files are excluded.

Generated files are written to the correct location.

Phase 5: AI Analysis Layer

Tasks:

1. Create AI provider abstraction.

2. Implement structured prompt templates.

3. Implement Repo Summary Agent.

4. Implement User Intent Extraction Agent.

5. Implement Feature Inventory Agent.

6. Implement Cross-Repo Comparison Agent.

7. Implement Best Practice Audit Agent.

8. Validate all AI outputs with schemas.

Tests:

Schema validation.

Mock AI responses.

Invalid AI response handling.

Agent output persistence.

Phase 6: Merge Plan UI

Tasks:

1. Display repo summaries.

2. Display feature matrix.

3. Display recommended base repo.

4. Display conflicts.

5. Display keep/adapt/rewrite/discard/create-new decisions.

6. Add approve plan button.

Tests:

Merge plan renders.

Approval state changes.

Missing analysis states handled.

Phase 7: Build Engine

Tasks:

1. Create final project workspace.

2. Apply base repo files.

3. Copy/adapt selected files according to merge plan.

4. Generate missing scaffold files.

5. Harmonize dependencies.

6. Generate docs.

7. Generate tests.

8. Create build task log.

Tests:

Output workspace created.

Docs generated.

Merge decisions map to output files.

Build task statuses update.

Phase 8: Validation and Repair

Tasks:

1. Detect available commands.

2. Run install where safe.

3. Run lint if available.

4. Run typecheck if available.

5. Run tests if available.

6. Run build if available.

7. Capture output.

8. Implement repair loop with max attempts.

9. Save failed outputs.

Tests:

Command detection.

Test result parsing.

Failed command logging.

Repair attempt logging.

Phase 9: Code Review and Final Export

Tasks:

1. Implement Code Review Agent.

2. Generate review markdown files.

3. Generate KNOWN_ISSUES.md if needed.

4. Generate zip export.

5. Add final results screen.

Tests:

Review files generated.

Zip contains final project.

Known issues generated when validation fails.

---

28. Required File Outputs from GitMash

Every GitMash build should produce this minimum output:

```
final-project/
  README.md
  PROJECT_SPEC.md
  ARCHITECTURE.md
  MERGE_PLAN.md
  DECISIONS.md
  TESTING.md
  DEVLOG.md
  AGENT_HANDOFF.md
  KNOWN_ISSUES.md if needed
```

---

29. Example Homepage Copy

GitMash

Turn scattered GitHub repos into one better project.

Paste 2 to 3 related GitHub repos, describe what you want to preserve, and GitMash will analyze, compare, harmonize, test, review, and document a new unified build.

Repo 1
[GitHub URL]

Repo 2
[GitHub URL]

Repo 3 Optional
[GitHub URL]

What should GitMash preserve or improve?
[Describe what you like from each repo and what you want the final project to become.]

[Analyze Repos]

---

30. Example User Brief Prompt

Use this helper text in the UI:

Tell GitMash what matters.

Examples:
- Repo 1 has the best backend and auth.
- Repo 2 has the best UI and dashboard.
- Repo 3 has the best AI workflow.
- Keep the final app in Next.js.
- Rewrite anything messy.
- Add tests and dev logs.
- Make it easy for another AI agent to continue.

---

31. Example Final Output Summary

GitMash Final Build Summary

Final Project

`unified-dashboard-ai`

Source Repos

1. `dashboard-v1`
2. `dashboard-v2`
3. `ai-dashboard-experiment`

Base Repo Selected

`dashboard-v2`

Why

It had the cleanest Next.js structure, strongest UI, and most complete dashboard flow.

Preserved

- Supabase auth from `dashboard-v1`
- Dashboard shell from `dashboard-v2`
- AI workflow concept from `ai-dashboard-experiment`

Rewritten

- AI workflow service
- Auth integration layer
- Dashboard data loading

Discarded

- Duplicate pages
- Old CSS
- Incomplete API routes

Validation

- Typecheck: passed
- Unit tests: passed
- Build: passed
- Code review: passed with minor recommendations

Docs Generated

- README.md
- PROJECT_SPEC.md
- ARCHITECTURE.md
- MERGE_PLAN.md
- DECISIONS.md
- TESTING.md
- DEVLOG.md
- AGENT_HANDOFF.md

---

32. Definition of Done

GitMash MVP is done when:

1. A user can submit 2 to 3 public GitHub repos.

2. The app validates and clones the repos.

3. The app filters and analyzes files.

4. The app generates repo digests.

5. The app generates structured AI analysis.

6. The app creates a merge plan.

7. The user can approve the plan.

8. The app generates a final output project.

9. The app generates tests where supported.

10. The app runs validation commands where supported.

11. The app performs AI code review.

12. The app writes dev logs.

13. The app creates handoff documentation.

14. The user can download the final output.

15. If anything fails, the app clearly documents what failed and why.

---

33. Strategic Recommendation

Build GitMash in two serious phases.

Phase 1: Analysis and Merge Plan

This is the must-win foundation.

The system must become excellent at:

understanding repos

comparing repos

extracting user intent

creating trustworthy merge plans

documenting decisions

If this is weak, the generated build will be weak.

Phase 2: Build and Repair

Once the analysis is strong, add:

project generation

tests

validation

repair loops

code reviews

export

This keeps the product from becoming a flashy but unreliable code-masher.

---

34. Codex Starting Prompt

Give Codex this prompt after sharing the PRD:

You are building GitMash according to the attached PRD.

Start by implementing the MVP in phases.

Do not attempt to build the entire product in one pass.

First create the app scaffold, data models, homepage input flow, repo URL validation, and local workspace structure.

Use TypeScript. Keep files small and modular. Add tests for URL parsing, repo validation, workspace path generation, and file filtering.

Every major implementation step must update DEVLOG.md.

Every architectural decision must update DECISIONS.md.

Do not skip documentation.

Do not claim a phase is complete unless the relevant tests pass.

When uncertain, implement the simplest reliable version that supports the PRD and document the tradeoff.

Key Takeaway

GitMash should be built around this promise:

> Paste related repos, describe what you want to keep, and GitMash creates a tested, reviewed, documented path to one better project.

The killer feature is not just code generation. The killer feature is the analysis pipeline plus dev discipline: preserving the best parts, harmonizing the architecture, testing along the way, reviewing the work, and documenting enough that any human or AI agent can continue from there.